

期末汇报

PPT 大纲（10 分钟 / ~12-15 页）

开场（1 页，30s）

- 项目名称 + 一句话定位：「基于动态知识图谱的回忆录访谈规划 Agent」
- 校企合作背景（腾讯）、团队 5 人
- 核心痛点：LLM 直接访谈 → 遗忘主线 / 提问浅 / 逻辑跳

一、我们做了什么（2 页，1.5min）

Page 1 — 系统全景图（一页讲清楚）

画一张从左到右的架构图，5 个核心模块用箭头串起来：

代码块

- 1 用户回答
- 2 → 实体提取 Agent（自由提取 Event/Person/Location/Emotion/Insight）
- 3 → 图谱写入（Neo4j + FAISS 向量去重）
- 4 → 三路混合检索（向量0.5 + 图遍历0.3 + 全文0.2 → RRF融合）
- 5 → 决策上下文构建（覆盖率 + 焦点叙事 + 图间隙 + 情绪 + 画像）
- 6 → Planner-Interviewer 统一 Agent（策略分析 + 生成问题，一次 LLM 调用）

Page 2 — Live Demo 截图 / 录屏

- 前端：Cytoscape 图谱可视化 + 覆盖率仪表盘 + 实时访谈面板
- 标注：主题覆盖率实时变化、实体节点动态增长

二、如何设计的（3 页，2min）

Page 3 — 三层记忆架构

层	存储	更新频率	作用
短期	SessionState.tr anscript	每轮同步	直接对话上下文
长期	Neo4j 知识图谱 + FAISS	每轮写入	覆盖率/焦点/探 索方向
长期	DynamicProfile 动态画像	每3-5轮异步	画像指导（性 格/家庭/价值 观）

强调：跨会话连续性 — 同一老人重新访谈时自动加载历史图谱

Page 4 — Planner 决策机制（核心创新）

- Planner 和 Interviewer 合并为一次 LLM 调用（减少延迟、避免信息损失）
- 内部 6 步推理链：语义理解 → 阶段判断 → 完整度评估 → 情绪感知 → 图谱判断 → 候选动作打分
- 8 种动作可选：continue_overview / deep_dive / clarify / switch_theme / move_to_person / reflection / end 等
- 输出结构化 planner_plan（可观测、可调试、可实验）

Page 5 — 检索策略：Schema-on-the-fly + RRF 三路融合

- 不预设固定槽位 → LLM 自由提取实体和关系（Schema-on-the-fly）
- 三路检索互补：向量（语义近似）+ 图遍历（关系发现）+ 全文（关键词命中）
- 实体去重：向量 cos > 0.85 + 名称子串双重校验
- 事件自动关联 McAdams 23 个生命主题（向量匹配）

三、探索历程（3 页，2min）

这是体现研究深度的部分，用时间线展示。

Page 6 — 架构演进时间线

代码块			
1	2月	基线搭建	Baseline Agent + 嵌入/LLM 评判
2	3月	槽位探索	固定8槽位 → 5槽位 → 6层事件提取
3		图谱可视化	NetworkX → 前端Cytoscape
4		合并引擎	GraphProjector + MergeEngine + MemoryCapsule
5	4月	Planner统一	分离的 Planner+Interviewer → 合并为单次调用
6		RAG引入	FAISS EventVectorStore + 语义记忆检索
7		动态画像	ProfileProjector 异步更新
8		Neo4j迁移	NetworkX → Neo4j 图数据库
9		批量测试	BatchInterview Pipeline + 计时
10	5月	GraphRAG重构	删除全部旧管线，全面切换 GraphRAG

11	智谱Embedding	本地模型 → 云端 embedding-3 (2048维)
12	实体合并	LLM 自主决定 new/update/skip
13	性能优化	get_graph_state 2.1s → 9ms

Page 7 — 关键取舍（保留 vs 舍弃）

舍弃（已删除）	原因	替代方案
固定槽位机制 (8→5→6维度)	信息丢失严重，强制结构化	Schema-on-the-fly 自由提取
NetworkX 内存图	无法持久化、不可扩展	Neo4j 图数据库
Planner + Interviewer 分离调用	两次 LLM 调用延迟高、信息损失	统一为单次调用 + 内部推理链
MemoryCapsule + GraphSummary	槽位依赖、维护复杂	GraphRAGDecisionContext 直接从 Cypher 构建
GraphProjector + MergeEngine	与槽位紧耦合	GraphWriter 向量去重 + LLM merge_action
本地 sentence-transformers	384维，中语义能力弱	智谱 embedding-3 (2048维)
CAMEL 框架依赖	过重，灵活性不足	纯 OpenAI SDK + 自定义 Tool Call
自适应路由校准 (routing_calibration)	复杂但效果不显著	简化为 LLM 内部候选动作打分

保留并持续优化：

- McAdams 23 生命主题体系
- 动态画像 (ProfileProjector)
- FAISS 向量索引 + RRF 混合检索
- Planner Tool Call（10 个工具，LLM 可自主调用查证据）
- 跨会话记忆桥接 (SessionGraphBridge)
- BatchInterview 批量实验管线

Page 8 — 技术亮点一句话总结

「从规则驱动到数据驱动，从槽位填充到图谱生长」
— 最大的架构决策是把 Planner 决策逻辑从硬编码规则（权重打分）转变为 LLM 内部推理 + 图谱证据驱动的 emergent behavior。

四、成果展示与数据对比（3 页，3min）

Page 9 — 对比实验设计

- Baseline：纯 LLM 对话，无 Planner / 无图谱

- GraphRAG Planner：完整管线（图谱构建 + 混合检索 + 决策上下文）
- 评估维度：话题覆盖率、对话深度、信息增益、非冗余度

Page 10 — 定量结果

用图表展示（从 `results/` 目录的数据）：

指标	Baseline	GraphRAG Planner	提升
主题覆盖率	?	?	?
平均信息增益/轮	?	?	?
问题非冗余率	?	?	?
实体提取数量	?	?	?

你需要从 `results/baseline-vs-planner/` 的数据中填入具体数字

Page 11 — 定性案例对比

选 1-2 段对话对比：

- Baseline：机械追问、重复提问、遗漏人生阶段
- Planner：自然过渡、主动跳转未覆盖主题、深挖情感细节

五、总结与展望（2 页，1min）

Page 12 — 项目成果总结

- 完整的 GraphRAG 访谈系统（4 个 Agent + 5 层服务 + 前后端）
- 可观测的 Planner 决策（`planner_plan` 结构化输出）
- 100+ 次 Git 提交，删除 13000+ 行旧代码的持续重构
- 基于腾讯真实样本的评估验证

Page 13 — 未来方向

- 多模态：照片引导叙事
- 真人访谈验证（目前是 User Simulator）
- 个性化大纲（从通用模板 → 根据老人背景定制）
- 回忆录生成（从访谈记录 → 自动成书）

时间分配建议

部分	页数	时间
开场	1	30s
做了什么	2	1.5min
如何设计	3	2min
探索历程	3	2min
数据对比	3	3min
总结展望	2	1min

系统架构全景

一、系统定位

这是一个基于 **GraphRAG** 的智能叙事访谈系统，核心目标是辅助深度人生回忆录访谈。系统通过实时构建个人知识图谱 + 语义检索 + 动态画像，驱动 LLM 生成有策略的访谈问题，确保访谈覆盖完整人生阶段并深挖高价值记忆。

二、全局数据流（一轮对话的完整生命周期）

代码块

```
1  用户回答 (WebSocket)
2  |
3  ▼
4  SessionOrchestrator.process_user_response()
5  |
6  |—— 1. GraphExtractionAgent.extract()           ← LLM 提取实体/关系
7  |    |—— 输入: 当前轮对话 + 近 3 轮上下文 + Neo4j 已有实体摘要
8  |    |—— 输出: GraphExtraction (entities, relationships, open_loops,
    |    emotional_state)
9  |
10 |—— 2. GraphWriter.write_extraction()           ← 写入图谱
11 |    |—— 实体嵌入 (EmbeddingService)
12 |    |—— 向量去重 (EntityVectorStore, cos > 0.85)
13 |    |—— Neo4j 写入节点 + 边
14 |    |—— 事件-主题自动关联 (向量匹配 Topic)
15 |    |—— 同步 FAISS 向量索引
16 |
17 |—— 3. HybridRetriever.retrieve()             ← 三路混合检索
18 |    |—— 通道 A: FAISS 向量搜索 (α=0.5)
19 |    |—— 通道 B: Neo4j 2-hop 图遍历 (β=0.3)
20 |    |—— 通道 C: Neo4j 全文搜索 (γ=0.2)
21 |    |—— RRF 融合排序 → prompt_text
```

```
22 |
23 |—— 4. GraphRAGDecisionContextBuilder.build()      ← 构建决策上下文
24 |    |—— 主题覆盖率 (Cypher 查询)
25 |    |—— 当前焦点叙事 (narrative_fragments)
26 |    |—— 图间隙 → 可探索方向
27 |    |—— 情绪状态 (transcript 内联推断)
28 |    |—— 低信息连续轮次计数
29 |    |—— 跨会话历史摘要
30 |    |—— 动态画像提示
31 |
32 |—— 5. InterviewerAgent.generate_question()      ← LLM 生成下一问
33 |    |—— System Prompt: 行为准则 + Planner 内部工作流 + 输出格式
34 |    |—— User Prompt: 画像 + 对话 + 焦点 + 覆盖度 + 线索 + 记忆脉络 + 策略提示
35 |    |—— Planner Tools (可选): get_theme_coverage,
    |    |    assess_event_completeness 等
36 |    |—— 输出: {planner_plan, action, question}
37 |
38 |—— 6. [异步] ProfileProjector.update_profile() ← 动态画像更新 (每 3-5 轮)
39 |
40 |—— 7. EvaluatorAgent.evaluate_turn()      ← 轮次评估 (同步)
41 |    |—— 信息增益: 实体数 × 0.15
42 |    |—— 非冗余度: SequenceMatcher 与近 3 轮问题比较
43 |    |—— 问题质量 = 信息增益×0.45 + 非冗余×0.35 + 覆盖增益×0.20
```

三、Agent 体系

Agent	文件	职责	LLM 模型
InterviewerAgent	interviewer_agent.py	生成下一访谈问题 + 内部 Planner 决策	INTERVIEWER_MODEL_NAME (默认 kimi-latest)
IntervieweeAgent	interviewee_agent.py	模拟老人回答 (User Simulator) ，支持 Tool Call 检索记忆	INTERVIEWEE_MODEL_NAME
GraphExtractionAgent	graph_extraction_agent.py	从对话中提取实体和关系	EXTRACTOR_MODEL_NAME (默认 moonshot-v1-8k)
EvaluatorAgent	evaluator_agent.py	评估每轮访谈质量 (纯规则，无 LLM)	无

四、检索系统（三路混合）

4.1 HybridRetriever — RRF 融合检索

位于 hybrid_retriever.py

通道	数据源	权重	机制
向量搜索	FAISS (内存)	$\alpha=0.5$	query $\rightarrow$ EmbeddingService 编码 $\rightarrow$ 内积搜索
图遍历	Neo4j	$\beta=0.3$	以 Top-K 向量命中为种子, 2-hop 展开
全文搜索	Neo4j fulltext index	$\gamma=0.2$	关键词匹配

融合算法: **Reciprocal Rank Fusion** (RRF, K=60)

代码块

```
1 score(entity) =  $\alpha/(K+\text{rank\_vector}) + \beta/(K+\text{rank\_graph}) + \gamma/(K+\text{rank\_fulltext})$ 
```

输出: 排序后的 `RankedEntity` 列表, 格式化为 prompt 文本 (max ~400 tokens)

4.2 EntityVectorStore — 多类型实体 FAISS 索引

位于 entity_vector_store.py

- 支持 5 种实体类型: Event / Person / Location / Emotion / Insight
- 内存 FAISS IndexFlatIP (内积 = L2 归一化后的余弦相似度)
- 支持按 entity_type 过滤检索
- 跨会话启动时由 `SessionGraphBridge` 预加载历史实体

4.3 EmbeddingService — 统一嵌入服务

位于 embedding_service.py

Provider	模型	维度
local	paraphrase-multilingual-MiniLM-L12-v2	384
openai	智谱 embedding-3 / OpenAI text-embedding-3-small	2048 / 1536

Provider 不可用时自动降级为**确定性词袋哈希嵌入** (blake2b)。

五、记忆架构（三层）

5.1 短期记忆: transcript (SessionState)

- 存储位置: models.py → `SessionState.transcript`
- 每轮同步追加 `TurnRecord`
- Planner 读取最近 3-5 轮作为直接上下文
- 包含: interviewer_question, interviewee_answer, extraction_result, turn_evaluation, debug_trace

5.2 长期记忆: Neo4j 图谱

- **实体节点:** Event / Person / Location / Emotion / Insight
- **关系边:** PARTICIPATES_IN / LOCATED_AT / TRIGGERS / RELATES_TO 等 (自由提取)
- **主题节点:** Topic, 通过 INCLUDES 关联到 Event
- **写入流程:** GraphExtraction → 向量去重 ($\cos > 0.85$) → Neo4j upsert + FAISS 同步
- **Schema-on-the-fly:** 不预设固定槽位, LLM 自由提取
- 实体 ID 生成: `{type}_{sha256(namespace:type:name)[:12]}` (确定性)

5.3 长期记忆: DynamicProfile (动态画像)

- 存储位置: `SessionState.dynamic_profile` → `DynamicElderProfile`
- 由 profile_projector.py 维护
- **更新频率:** 每 3-5 轮异步更新一次
- **4 个维度:**
 - core_identity_and_personality (身份/性格/说话风格)
 - current_life_status (日常/兴趣/健康)
 - family_situation (婚姻/子女/兄弟姐妹)
 - life_views_and_attitudes (人生观/价值观)
- 每个字段带 confidence + evidence_turn_ids
- 输出 planner_guidance 列表指导 Planner

5.4 跨会话记忆: SessionGraphBridge

位于 session_graph_bridge.py

- 通过 `elder_id` (name + birth_year) 关联同一老人的不同访谈会话
- 启动新会话时:
 - a. 查询 Neo4j 历史实体/关系

- b. 预加载 EntityVectorStore
- c. 生成历史摘要文本注入开场 prompt
- d. 提取未闭合线索 (open_loops) 作为追问方向

5.5 IntervieweeAgent 的记忆系统: ElderMemorySystem

位于 elder_tools.py

- 从 JSON 老人档案加载结构化记忆 (按人生阶段分 period)
- 提供 6 个 Tool Call 工具:
 - `search_memories_by_keywords` — 关键词搜索
 - `search_memories_by_tags` — 标签搜索
 - `get_memory_by_id` — 精确获取
 - `get_memories_by_period` — 按时期获取
 - `get_related_memories` — 关联记忆
 - `search_memories_by_semantic` — FAISS 语义搜索
- IntervieweeAgent 通过 OpenAI Tool Call 机制自主决定何时检索记忆

六、Prompt 组成

6.1 InterviewerAgent 的 System Prompt

位于 interviewer_agent.py:122-267 硬编码

代码块

```
1 | _____|
2 | | System Prompt |
3 | | |—— 角色定位：充满好奇心的传记访谈者 |
4 | | |—— 访谈覆盖主题（Life Story Interview 规程） |
5 | | |—— 人生篇章（划分阶段） |
6 | | |—— 关键场景（高光/低谷/转折/童年/智慧/温情） |
7 | | |—— 重要人物 |
8 | | |—— 价值观与信仰 |
9 | | |—— 未来展望 |
10 | |—— 行为准则（5 条） |
11 | | |—— 先回应后提问 |
12 | | |—— 好奇多于礼貌 |
13 | | |—— 问题类型灵活选择 |
14 | | |—— 抓住一个切入点 |
15 | |—— 隐藏技术细节 |
```

```

16 | | | — 内部 Planner 工作流 (6 步)
17 | | | | — 1. 语义理解
18 | | | | — 2. 访谈阶段判断
19 | | | | — 3. 当前事件完整度判断
20 | | | | — 4. 情绪与精力判断
21 | | | | — 5. 图谱和主题判断
22 | | | | — 6. 候选动作比较 (8 种动作打分)
23 | | | — 判停与结束判断
24 | | — 输出要求 (严格 JSON)
25 | | — 工具调用说明
26 | —————

```

6.2 InterviewerAgent 的 User Prompt (动态组装)

位于 interviewer_agent.py:269-375

代码块

```

1 |
2 | User Prompt (每轮动态生成)
3 |
4 | ## 1. 受访者基本信息
5 |     name / birth_year / hometown / background
6 |
7 | ## 2. 已了解的受访者特点 (动态画像)
8 |     core_identity / life_status / family / views
9 |     + planner_guidance 建议
10 |
11 | ## 3. 最近对话 (3-5 轮)
12 |     问/答 对
13 |
14 | ## 4. 正在聊的经历
15 |     focus_rich_text + 关联人物 + 关联地点 + 情感线索
16 |
17 | ## 5. 人生故事覆盖情况
18 |     overall_coverage + 已丰富主题数 + 稀缺方向数
19 |     + 当前焦点方向
20 |
21 | ## 6. 待探索的话题线索
22 |     图间隙分析 → 角度列表
23 |
24 | ## 7. 叙事记忆脉络
25 |     HybridRetriever 的 RRF 融合结果
26 |
27 | ## 8. 策略提示
28 |     低信息连续轮次警告
29 |     跨会话历史 + 未展开线索

```

```
30 |  
31 | ## 9. 你的任务  
32 |     任务指令 + 记住事项 + JSON 格式要求  
33 |
```

6.3 Planner Tool Call (可选工具)

位于 planner_tools.py，共 10 个工具:

工具	用途
get_theme_coverage	主题覆盖率 + 稀缺主题
get_current_focus_snapshot	当前焦点 + 人物地点 + 线索
assess_event_completeness	8 维度完整度评估 (time/location/people/sequence/cause/result/feeling/reflection)
retrieve_related_context	GraphRAG 上下文 + 跨会话线索
detect_open_loops_and_conflicts	未闭合线索 + 低信息提示
get_entity_context	图谱实体 N-hop 关联
query_graph_entities	按类型/文本搜索图谱
detect_graph_patterns	检测重复模式
get_graph_summary	图谱统计
get_theme_detail	主题详情

6.4 IntervieweeAgent 的 System Prompt

由 elderly_promot.py 通过 Jinja2 模板生成:

代码块

```
1 |  
2 | Jinja2 模板 → 老人 System Prompt  
3 | ┌─ 核心身份与性格 (name/age/gender/hometown/...)  
4 | ┌─ 当前生活状况 (routine/hobbies/health/...)  
5 | ┌─ 家庭情况 (marital/children/grandchildren/...)  
6 | ┌─ 人生观点与态度 (past/present/family/society)  
7 | ┌─ 人生阶段概述 (按 period 遍历)  
8 | ┌─ 敏感话题 (topic + trigger_keywords)  
9 | ┌─ 知识边界 (medical/tech/history/policy)  
10 | ┌─ 记忆管理机制说明  
11 | ┌─ 回答原则 (7 条)  
12 | ┌─ JSON 输出格式 (thoughts + reply)  
13 | └─ 重要提醒
```

6.5 GraphExtractionAgent Prompt

位于 `src/prompts/graph_extraction_prompt_v1.md` (外部文件, 运行时加载)

输入 JSON:

代码块

```
1  {
2    "current_turn": {"interviewer": "...", "respondent": "..."},
3    "context": [最近 3 轮],
4    "existing_graph_context": [Neo4j 已有实体摘要 (最多 60 个)]
5  }
```

输出 JSON:

代码块

```
1  {
2    "has_content": true,
3    "entities": [{"entity_type": "Event|Person|...", "name": "...",
4    "description": "...",
5    "merge_action": "new|update|skip", "merge_target_id": "..."}],
6    "relationships": [{"source_name": "...", "target_name": "...",
7    "relation_type": "..."}],
8    "narrative_summary": "...",
9    "open_loops": [...],
10   "confidence": 0.85
11 }
```

七、Planner 输出格式

InterviewerAgent 返回的 JSON:

代码块

```
1  {
2    "planner_plan": {
3      "stage": "life_overview | event_deepening | theme_expansion | reflection |
4      closing",
5      "selected_action": "continue_life_overview | deep_dive_event | clarify |
6      confirm_summary | switch_theme | move_to_person | move_to_period |
```

```

gentle_reflection | end",
5     "focus": {"type": "life_period|event|person|location|theme|reflection",
"label": "简短中文焦点"},
6     "event_completeness": {"score": 0.0-1.0, "missing_dimensions": [...],
"reason_summary": "..."},
7     "emotion_signal": {"energy": 0.0-1.0, "valence":
"positive|neutral|negative", "support_needed": bool},
8     "candidate_actions": [{"action": "...", "score": 0.0-1.0,
"reason_summary": "..."}],
9     "selected_slot_or_angle":
"life_timeline|time|location|people|sequence|cause|result|feeling|reflection|th
eme|person",
10    "tone":
"respectful_warm|curious_gentle|empathetic_supportive|light_conversational",
11    "question_intent": "一句话意图"
12 },
13 "action": "continue | next_phase | end",
14 "question": "你的访谈问题"
15 }

```

八、核心数据模型

位于 models.py + narrative_models.py

代码块

```

1  SessionState
2      |— session_id, mode, created_at, updated_at
3      |— elder_profile: ElderProfile (name, birth_year, age, hometown,
background_summary)
4      |— transcript: List[TurnRecord]
5          |— turn_id, turn_index, timestamp
6          |— interviewer_question, interviewee_answer
7          |— extraction_result: ExtractionResult
8              |— metadata + graph_delta
9              |— fragment_candidates: List[NarrativeFragment]
10             |— graph_extraction: GraphExtraction
11             |— entities: List[ExtractedEntity]
12             |— relationships: List[ExtractedRelationship]
13             |— open_loops, emotional_state, confidence
14             |— turn_evaluation: TurnEvaluation
15             |— debug_trace (extraction_ms, write_ms, retrieval_ms, planner_plan,
...)
16         |— theme_state: Dict[str, ThemeState]
17         |— dynamic_profile: DynamicElderProfile

```

```
18 |— narrative_fragments: Dict[str, NarrativeFragment]
19 |— pending_question, pending_action
20 |— evaluation_trace: List[TurnEvaluation]
21 |— session_metrics: SessionMetrics
22 |— metadata: Dict (pending_planner_plan, fallback_repeat_count, ...)
```

九、技术栈与基础设施

层	技术
前端	React + TypeScript + Vite + Cytoscape.js (图谱可视化)
后端 API	FastAPI + WebSocket (端口 8000)
对比调试	Flask + SSE (端口 9999)
图数据库	Neo4j 5 Community (Docker, 端口 7687)
向量索引	FAISS IndexFlatIP (内存)
LLM	Kimi/Moonshot (OpenAI 兼容) — 多模型分工
嵌入模型	智谱 embedding-3 (云端) / paraphrase-multilingual-MiniLM-L12-v2 (本地)
配置	.env + Config 类 (环境变量)

LLM 模型分工

角色	配置项	默认模型
访谈问题生成	INTERVIEWER_MODEL_NAME	kimi-latest
受访者模拟	INTERVIEWEE_MODEL_NAME	kimi-latest
图谱提取	EXTRACTOR_MODEL_NAME	moonshot-v1-8k
结构化输出	STRUCTURED_MODEL_NAME	moonshot-v1-8k

十、关键设计决策

- Schema-on-the-fly:** 不预设固定槽位，LLM 自由提取实体和关系
- Planner-NLG 解耦:** Planner 输出结构化指令 (planner_plan + action + question)，不直接说话
- RRF 三路融合:** 向量 + 图 + 全文，互补增强检索召回
- 实体去重:** 向量余弦 > 0.85 + 名称子串匹配双重校验

- 5. 异步画像更新: 动态画像每 3-5 轮后台线程更新，不阻塞主流程
- 6. Tool Call 双向: InterviewerAgent 有 Planner Tools, IntervieweeAgent 有 Memory Tools
- 7. 跨会话连续性: 通过 elder_id 关联同一老人，预加载历史图谱 + 向量索引

项目探索历程

时间跨度: 2026-02-05 ~ 2026-05-08 (约 3 个月)
总提交: ~103 次 | 分支: 14 个 | 贡献者: 7 人

第一周：地基搭建期（2月5日 - 2月6日）

分支: main / origin/judgetest / origin/user_rag

日期	提交	内容
2月5日	340bbd44	最初的 embedding 和 LLM 评测对比脚本
2月5日	3f20b7f4	user_rag 分支: RAG 向量检索的早期探索
2月6日	a98bcd3e	Baseline Agent 诞生 : 无 Planner 的纯 LLM 访谈 Agent
2月6日	9fa4db1b	评估判据改进

核心决策: 先做 Baseline（无 Planner），建立对比基准。早期就在探索用 embedding 做语义匹配还是用 LLM 直接判断，结论是两者都需要。

保留: Baseline Agent 作为对比对照组一直保留至今
舍弃: user_rag 分支的早期向量检索方案（后续被 FAISS + Neo4j 取代）

第二周：原型验证期（3月5日 - 3月15日）

分支: origin/user-simulation / origin/full-version / origin/basic-navigator

日期	提交	内容
3月5日	f87fa198	User Simulator（受访者模拟器）开始开发
3月12日	4f37ba2c	Flask 前端 Demo
3月12日	f9170aa2	Interviewer Agent + 多轮对话 Demo
3月12日	7bfa8936	basic-navigator 分支：基础导航策略
3月13日	8.85E+10	前端可视化 ：动态事件图谱（Cytoscape.js）
3月13日	7fd190e9	人物思维导图 + 看板样式
3月15日	7e84ae1b	放弃 CAMEL 包 ：重构 base_agent，去掉外部依赖

核心探索：

- 尝试了 **CAMEL-ai** 框架做多 Agent 对话 → 效果不好/太重，**手动实现** Agent 通信
- 前端用 Cytoscape.js 做图谱可视化 → 效果好，一直保留
- `basic-navigator` 分支尝试简单的导航策略 → 后续被 Planner 内部工作流取代

保留: IntervieweeAgent 的 Tool Call 记忆系统、Cytoscape.js 可视化

舍弃: CAMEL 框架依赖、`basic-navigator` 分支的简单路由

第三周：槽位架构期（3月19日 - 3月20日）

分支: `feature/dynamic-event-graph`

日期	提交	内容
3月19日	8fff8ad1	槽位体系诞生 ：5 维槽位（time/location/people/event/reflection）+ 覆盖率计算
3月19日	b2e47399	动态事件图谱核心架构（NetworkX）
3月19日	d3bdd889	事件提取格式从 5 层扩展为 6 层，增加未展开线索
3月20日	6589d750	GraphProjector / MemoryProjector / MergeEngine 三件套
3月20日	359a49df	异步 Baseline 评估，对比调试界面
3月20日	d049c363	多模型配置（Interviewer / Interviewee / Extractor 分工）

核心策略：设计了完整的槽位填充机制

- 每个事件有固定槽位 → 计算填充率 → 覆盖率驱动下一步决策
- GraphProjector：基于老人画像初始化主题激活
- MemoryProjector：构建 MemoryCapsule（会话摘要 + 情绪状态）
- MergeEngine：新提取事件与已有 CanonicalEvent 合并

保留: 多模型分工的配置模式

舍弃: 整套槽位体系在 5/2 被彻底删除（ afd4f626 ），被自由提取取代

第四周：Agent 架构重构期（4月2日 - 4月3日）

日期	提交	内容
4月2日	0f4710b8	Tool Calling Tracker（追踪 Agent 工具调用）
4月3日	e577ee06	重大重构：Planner + Interviewer 合并为统一 Agent

关键决策：之前 Planner 和 Interviewer 是两个独立的 Agent/LLM 调用，合并为一个后：

- 一次 LLM 调用完成「分析 → 决策 → 生成问题」
- System Prompt 内置 Planner 工作流（6 步分析）
- 输出结构化 JSON: {planner_plan, action, question}

舍弃: 独立的 planner_agents.py / base_agents.py / streaming_interview_engine.py / prompt_formatter.py 全部删除

第五周：向量检索探索期（4月16日）

日期	提交	内容
4月16日	f9f8c222	EventVectorStore: FAISS 余弦相似度搜索
4月16日	2b929f5c	混合候选选择：规则 Top-2 + 向量 Top-2
4月16日	e61eea51	EventVectorStore 集成到 Session 生命周期
4月16日	66739cce	RAG 语义记忆检索设计文档
4月16日	7f71e423	IntervieweeAgent 增加语义记忆搜索 Tool
4月16日	d79d6b85	动态画像 (DynamicProfile): 4 维画像 + 异步更新

关键突破：

- 认识到纯规则不够，引入 FAISS 向量检索
- IntervieweeAgent 不再只用关键词搜索记忆，增加了语义搜索能力
- 动态画像替代静态 elder_profile，每 3-5 轮异步更新

保留: EntityVectorStore、DynamicProfile、语义记忆 Tool

舍弃: 旧的 event_vector_store.py （后来重构为 entity_vector_store.py ，支持多类型）

第六周：Neo4j + 图数据库期（4月17日）

日期	提交	内容
4月17日	75c901b7	Neo4j 数据模型和查询工具
4月17日	12b5d121	覆盖率缓存刷新 + 主题槽位同步到 Neo4j
4月17日	d67dd901	移除 NetworkX：全部迁移到 Neo4j
4月17日	2805c957	记忆提取 Prompt 优化
4月17日	1e15a3a5	自适应路由校准：三阶段校准 + 个性化配置

关键决策：

- 从内存图 (NetworkX) 迁移到持久化图数据库 (Neo4j)
- 支持跨会话记忆持久化

保留: Neo4j 作为核心存储

舍弃: NetworkX（d67dd901）、自适应路由校准（routing_calibration.py 在 5/2 被删除）

第七周：Pipeline 工程化期（4月23日 - 4月24日）

日期	提交	内容
4月23日	39d50693	Timer + TurnTiming 管道延迟追踪
4月23日	8767ad7c	批量访谈管道 + 每模块计时
4月23日	03cea01d	自动批量执行 + 计时
4月24日	e46527ec	批量测试管道日志

保留: 批量测试框架（scripts/）

舍弃: 旧的 pipeline/ 包在 5/2 被删除

第八周：GraphRAG 革命期（5月2日 - 5月3日） — 最大转折点

日期	提交	内容
5月2日	8ca0e612	GraphRAG 管线实现，支持与旧管线并行切换
5月2日	afd4f626	删除旧管线，全面切换到 GraphRAG
5月2日	1625105	修复 embedding fallback 的危险逻辑
5月3日	80dbee22	性能优化：get_graph_state 从 2.1s → 9ms
5月3日	10e53521	嵌入服务回退机制 + 单元测试

afd4f626 是项目最大的一次删除（73 个文件，删除 13,380 行，新增 2,562 行）：

被删除的 14 个核心文件：

- src/agents/extraction_agent.py — 旧的 8 槽位提取 Agent
- src/agents/planner_agent.py — 旧的独立 Planner Agent
- src/agents/planner_agents.py — 旧的 Planner Agent 组
- src/agents/prompt_formatter.py — 旧 Prompt 格式化器
- src/agents/streaming_interview_engine.py — 流式访谈引擎
- src/core/event_extractor.py — 旧事件提取器
- src/core/event_node.py — 旧事件节点（含槽位）
- src/core/graph_manager.py — NetworkX 图管理器
- src/core/theme_node.py — 旧主题节点
- src/orchestration/planner_decision_policy.py — 旧的 Planner 决策策略
- src/orchestration/routing_calibration.py — 自适应路由校准
- src/orchestration/turn_routing_policy.py — 轮次路由策略
- src/services/coverage_calculator.py — 旧的覆盖率计算器
- src/services/merge_engine.py — 旧的合并引擎
- src/services/graph_projector.py / memory_projector.py / neo4j_graph_projector.py
- src/pipeline/ 整个目录
- src/rag/ 整个目录（旧 RAG 模块）
- 前端 mockData.ts / graphTransformer.ts / PersonView.tsx

新增替代：

- GraphExtractionAgent — 自由格式提取（无固定槽位）
- HybridRetriever — 三路 RRF 融合检索

- GraphWriter — Neo4j 写入 + 向量去重
- GraphRAGDecisionContext — 图原生决策上下文
- EntityVectorStore — 多类型实体 FAISS 索引
- SessionGraphBridge — 跨会话记忆桥接
- 智谱 Embedding-3 替代本地 sentence-transformers

核心设计理念转变：

- 旧: 固定 8 槽位 → 填充率 → 规则驱动决策
- 新: 自由提取实体/关系 → 图谱生长 → 向量+图+全文混合检索 → LLM 驱动决策

第九周：打磨期（5月4日 - 5月8日）

日期	提交	内容
5月4日	5522c235	Planner Tool 集成增强（10 个只读工具）
5月7日	ef1909a6	计时功能 + SSE 前端展示
5月7日	9e9cdad9	Planner 会话报告 API + 前端报告抽屉
5月7日	8f45c704	实体类型图标和描述信息
5月8日	0f10c8d7	Prompt 优化 + 动态画像构建逻辑优化
5月8日	bcd66f55	实体合并操作支持（merge_action: new/update/skip）
5月8日	326eace0	Baseline vs Planner 对比大批量测试
5月8日	888604c0	查询优化

分支全景

分支	状态	探索方向	结论
main	活跃	主线开发	→
origin/user_rag	废弃	早期 RAG 向量检索	被 GraphRAG 取代
origin/judgetest	废弃	评估判据开发	合并到 main
origin/user-simulation	废弃	用户模拟器	合并到 main
origin/basic-navigator	废弃	简单导航策略	被 Planner 工作流取代
origin/full-version	废弃	全功能版本	合并到 main
feature/dynamic-event-graph	废弃	动态事件图谱 + 槽位	被自由提取取代
anoversion	废弃	另一版本探索	未采用
feature/graph-rag-pipeline	废弃	GraphRAG 并行测试	切换完成，已合并
origin/latest_with_logging	废弃	日志增强	已合并

策略演进总结

维度	第一版 (2-3月)	第二版 (4月中)	第三版 (5月，当前)
事件提取	固定 8 槽位填充	规则+向量混合候选	自由格式提取 (GraphExtractionAgent)
图存储	NetworkX (内存)	NetworkX → 迁移 Neo4j	Neo4j + FAISS 双写
检索	关键词匹配	FAISS 向量搜索	三路 RRF 融合 (向量+图+全文)
Planner 决策	独立 Agent	合并到 Interviewer	合并 + Tool Call 证据获取
覆盖率度量	槽位填充率	槽位填充率	叙事丰富度四维打分
记忆系统	无	EventVectorStore	EntityVectorStore (5 类型) + SessionGraphBridge
画像	静态 JSON	DynamicProfile	DynamicProfile + 异步更新
嵌入模型	本地 MiniLM	本地 MiniLM	智谱 Embedding-3 (云端 2048 维)
Pipeline	单次访谈	批量 + 计时	GraphRAG 全管线 + 指标监控

决策复盘（什么被舍弃了、为什么）

- CAMEL 框架 — 太重，控制力不够，手动实现 Agent 更灵活
- 固定槽位体系 — 太死板，老人回答千差万别，强行填槽导致信息丢失。自由提取更自然
- NetworkX 内存图 — 不持久化，重启丢失，无法跨会话。Neo4j 是正确选择
- Planner 与 Interviewer 分离 — 两次 LLM 调用延迟高、上下文割裂。合并为一次调用更好

- 5. 独立决策策略文件 (planner_decision_policy.py, turn_routing_policy.py) — 规则太多维护不动，交给 LLM 在 Prompt 内推理
- 6. 自适应路由校准 — 过度工程化，实际效果不明显
- 7. 旧 RAG 模块 (src/rag/) — 功能太简单，被 GraphRAG 三路融合取代
- 8. 本地 sentence-transformers — 维度低(384)、需下载模型、中英文效果一般。智谱云端 API 更好

Baseline vs Planner without GraphRAG vs Planner with GraphRAG

目前主要只有提取和回复生成这两个步骤会涉及 token 消耗和时间，本次针对这两个模块进行了小规模的测试。

- GraphRAG 显著增加了事件提取的 token 消耗，但是降低了模型输入的 token 消耗
- Planner without GraphRAG 相比 with GraphRAG 的回复更长导致回复时间更长，但是提取消耗的时间相差不大

Token 消耗	Baseline (3)	Planner Without GraphRAG (3)	Planner With GraphRAG (3)
总计	1415 / 987	15096 / 6576	23892 / 5521
平均	529 / 322	5032 / 2192	7964 / 1840
回复	1415 / 987	8022 / 4302	9540 / 1513
提取	NA	7074 / 2274	14352 / 4008

延迟	Baseline (3)	Planner Without GraphRAG (3)	Planner With GraphRAG (3)
总计	86.7	286.3	239.7
平均	28.9	95.4	79.9
回复	86.7	162.2	103.8
提取	NA	124.1	135.9

注

- 这里的模型输入也取决于受访者的回复长度，所以在大规模测试的时候会测试更多的受访者
- 目前采用的模型是 Kimi 2.5 ，后续计划探索更多模型的 token 消耗和延迟

